

3. DevOps és DevSecOps

92

- A XXI. Század szoftverfejlesztési gyakorlata – 2009. tekinthető a megszületése időpontjának, ekkor volt az első DevOpsDay.
- A DevOps szoftverfejlesztési gyakorlatok és eszközök összessége, melyek magába foglalják és automatizálják a szoftverfejlesztést (Dev) és az informatikai üzemeltetést (Ops) a szoftver teljes életciklusára.
- Ennek érdekében a fejlesztők és az üzemeltetők közösen dolgoznak, - azaz - mindenki felelős a termékért.

92

93

- Közös cél, hogy lerövidítsék a rendszerfejlesztési életciklust ugyanúgy, ahogy a funkciók javítását, továbbfejlesztését, a hibák elhárítását (mindkét oldalra hatással van).
- Gyorsabbá válik a termék szállítása – ezzel az értékteremtés - a felmerülő új igényeket, hibákat a fejlesztés, javítás után hamarabb kapja meg a felhasználó -> nő az ügyfél elégedettség és ezzel a cég piaci pozíciója és versenyképessége.
- Az üzemeltetői tapasztalatok már a fejlesztés során beépülhetnek a termékbe.

93

3.1. A DevOps elvei

94

1. Minden fázist (amit lehet) automatizáljunk, mert így
 - ❖ Hatékonyabbá tesszük a folyamatokat (automatikus kódgenerálás, tesztelés, integráció)
 - ❖ Kiküszöböljük az emberi hibákat
 - ❖ Javítjuk a minőséget (protokollok automatikus betartása)
2. Az értékteremtésben résztvevők közötti folyamatos kommunikáció (nem csak a fejlesztők között).
3. A fejlesztőcsapatok közötti leg szorosabb együttműködés a termék teljes életciklusa alatt.

94

A DevOps előnyei

95

- Hatékonyabb és gyorsabb termékfejlesztés (fejlesztés, tesztelés, kiadás) a folyamatos szállításnak köszönhetően.
- Folyamatos hibajavítás -> folyamatos aktualizálás (patch-ek, release-k)-> elégedettebb felhasználók, -> piaci siker
- Az olyan gyakorlatok, mint a folyamatos integráció és szállítás garantálja a változtatások működőképességét és biztonságosságát.
- A folyamatos monitorozás pedig a teljesítményről nyújt valós idejű információkat

95

3.2. DevOps gyakorlatok

96

Folyamatos integráció (CI)

- Korábbi integrációs stratégiákkal (kötött időközönként, nagyobb logikai egység elkészülte után) ellentétben a kódban történő változtatások vagy az új kódok automatikusan feltöltésre kerülnek (akár naponta vagy naponta több alkalommal is).
- Ezáltal az új build-ek azonnal elérhetővé válnak és gyorsan tesztelésre kerülnek. Ez lehetővé teszi a DevOps csapatoknak a feature-k gyorsabb elkészítését, a hibák gyorsabb javítását.
- Ezáltal javul a szoftver minősége, csökken az új szoftverfrissítések validálásához és kiadásához szükséges idő.

96

Folyamatos szállítás (CD)

97

- A folyamatos szállítás gyakorlatilag a folyamatos integráció kiterjesztésének tekinthető, mivel a kódmódosítások gyakran és automatikusan élekednek.
- Ennek következtében a felhasználók folyamatosan visszajelzéseket tudnak adni a legújabb verzióról, a fejlesztők pedig szükség szerint gyorsan tudnak módosítani vagy javítani a szoftveren.
- Így az elkészült szoftver a felhasználók valós idejű igényeit tudja kielégíteni, ami az ügyfél elégedettséget és így a versenyképességet nagy mértékben megnöveli.

97

Infrastruktúra, mint kód (IaC)

98

- Eddig az infrastruktúra leírások scriptekben valósultak meg.
- A DevOpsnál az IaC túlmutat az infrastruktúra-konfiguráció egyszerű szkriptelésén, mivel az infrastruktúra definícióit valódi kódként kezeli, kódellenőrzést, teszteket alkalmaz, illetve a definíciók tárolhatók és szükség szerint visszaállíthatók.
- Ennek előnye, hogy verzióváltozás esetén felmerülő problémáknál lehetőség van a visszaállításra, amely nagy biztonságot ad.
- Felhő-alapú rendszereknél - ahol bonyolult az infrastruktúra - nagyon előnyösen használható.

98

Automatizálás

99

- Az automatizálás lehetővé teszi a Dev és az Ops számára a folyamatok egyszerűsítését és a csapatok számára a nagyobb termelékenységét.
- Az automatizálás csökkenti a manuális műveletek számát, ezáltal javítja a szoftver fejlesztés és telepítés minőségét és biztonságát.
- A DevOps-automatizálás olyan technológiával egészül ki, amely csökkentett emberi segítséggel végez feladatokat olyan folyamatokban, amelyek megkönnyítik az üzemeltetési és fejlesztési csapatok közötti visszacsatolási hurkokat, így az iteratív frissítések gyorsabban telepíthetők a termelésben lévő alkalmazásokba.

99

3.3. A DevOps életciklus fázisai

100

A DevOps 8 fázisból áll:

- Plan – Tervezés
- Code – Kódolás
- Build – Építés
- Test – Tesztelés
- Release – Kiadás
- Deploy – Telepítés
- Operate – Működtetés
- Monitor – Monitorozás

A hagyományos szemlélettel szemben:

- Nincs külön analízis
- A tervezés mellett külön van építés
- A korábbiaknál kiemeltebb szerepe van a kódolásnak
- Nincs külön fázisa az integrációnak (CI)

Áthelyeződnek a hangsúlyok.

100

A DevOps életciklus működése

101

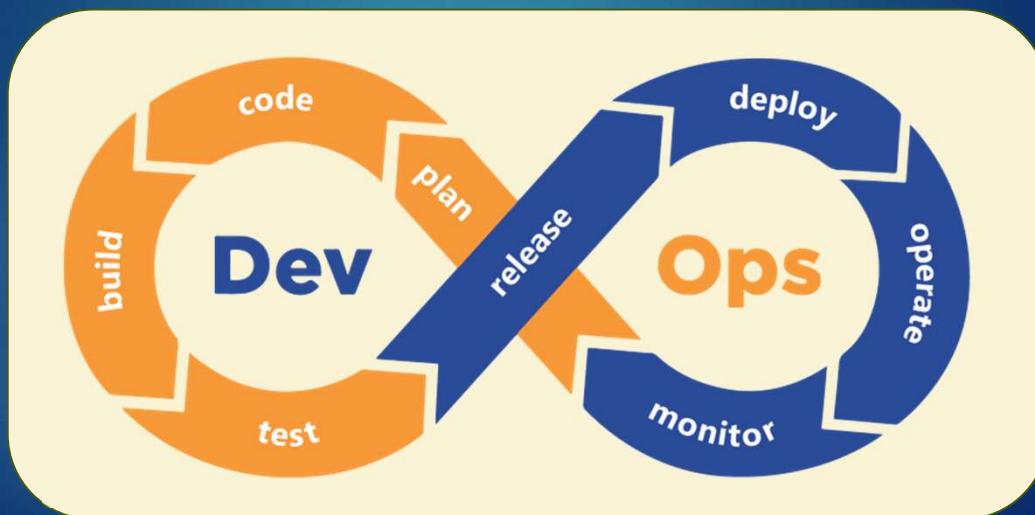


Alapvetően egy ciklikusan ismétlődő szekvenciális modell, mely két részből és 8 fázisból áll.

101

A DevOps életciklus fázisai másképp elrendezve

102



102

3.4. A DevOps-ot támogató eszközök

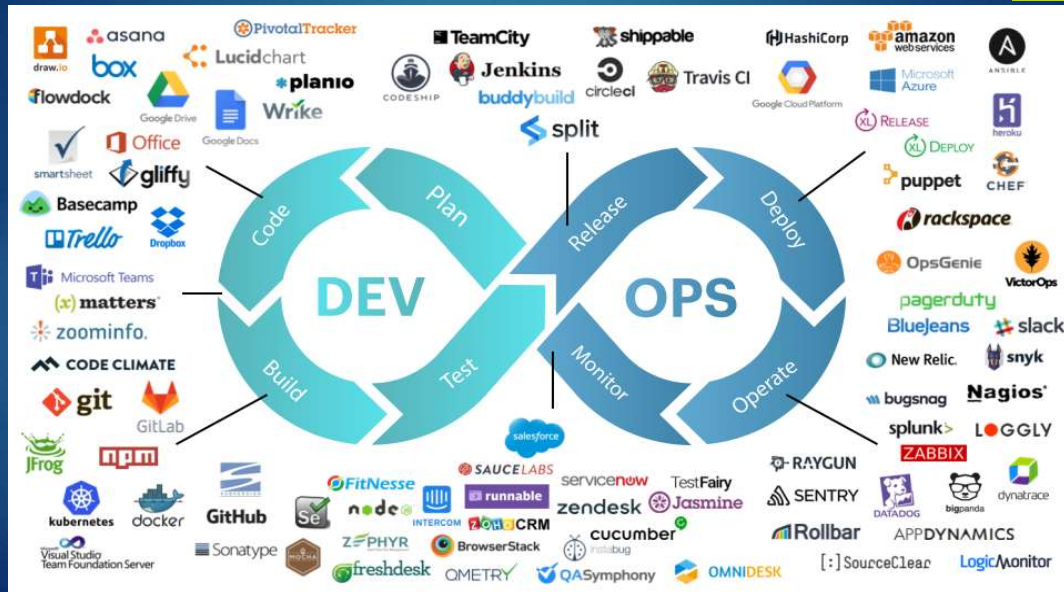
103

- **Verziókezelés, kódolás:** Git (GitLab, GitHub, Bitbucket), SVN, Jira
- **Build tool:** Maven
- **Release Tool:** Jenkins, Travis, Bamboo
- **Konfiguráció menedzsment:** Puppet, Ansible, Saltstack Chef
- **Container technológiák:** Docker, Kubernetes
- **Felhőalapú tárolás:** AWS, Azure, Google Cloud Platform
- **Monitorozás, riasztási és incidens-jelző rendszer:** Appdynamics,
- **Tesztelés:** Selenium, Gremling, Raygun, Spunk Cloud
- **Kommunikáció és együttműködés:** Slack
- **IT Ticketing:** Servicenow

103

A DevOps-ot támogató eszközök

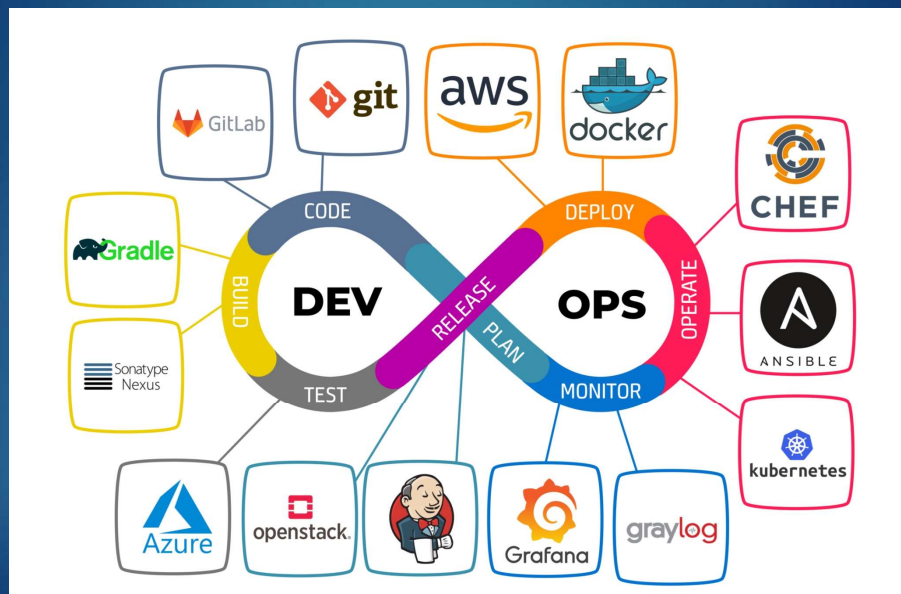
104



104

Kicsit leegyszerűsítve

105



105

3.5. DevSecOps (Development – Security - Operations)

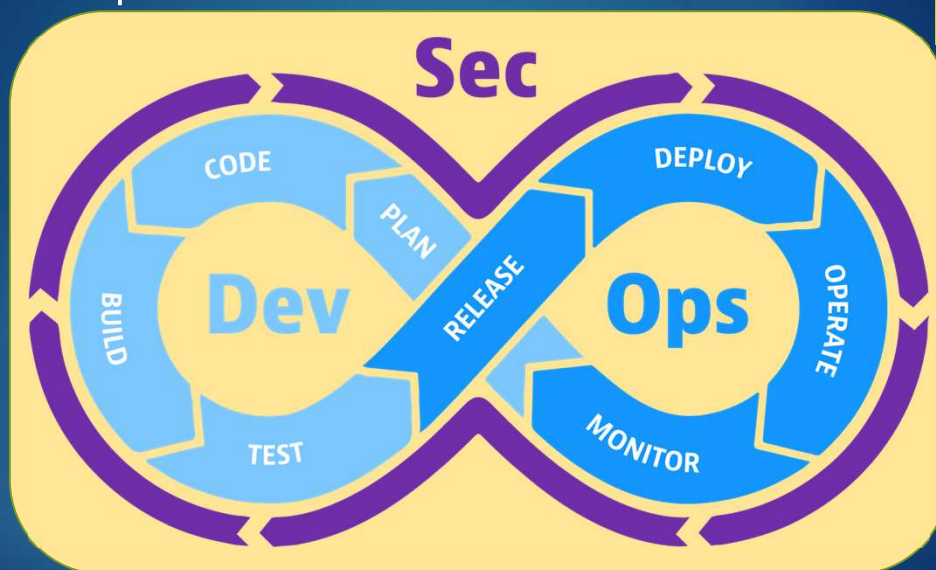
106

- Egy olyan keretrendszer, amely a biztonságot a szoftverfejlesztési életciklus minden fázisába integrálja.
- Cél hatékonyan csökkenteni a biztonsági réseket (sérülékenységeket) tartalmazó kód kiadásának kockázatát.
- A korábbi szemléletben a biztonság az utolsó fázisokra maradt (sérülékenység tesztelés)
- A DevSecOps esetében az együttműködés és az automatizálás révén a csapatok megosztják a biztonsággal kapcsolatos felelősséget (minden fázisban fókuszban van a security).
- A DevSecOps kifejezetten nagy felhő-alapú rendszerek biztonság-kritikus fejlesztési stratégiája (komplexitás miatt).

106

DevSecOps

107



Lásd a Microsoft korábbi módszerét az SDL-t

107

3.5.1. DevSecOps gyakorlatok

108

Folyamatos biztonság

- A DevOps-hoz képest a DevSecOps a folyamatos biztonsággal egészül ki, ami azt jelenti, hogy a megismert mindegyik fázisban nagy hangsúlyt fektetünk a security-ra.
- Ide értve:
 - ❖ a fenyegetések modellezését a folyamat korai szakaszában,
 - ❖ az automatizált biztonsági tesztelést a teljes életciklus során,
 - ❖ speciális security központú vizsgálatokat a tesztelés fázisában (pl. etikus hackkelés, fuzzolás)
 - ❖ a monitorozás során behatolás vizsgálatokat végzünk, különös figyelmet szentelünk az incidenseknek

108

„Shift left”

109

- A „shift left” egy DevSecOps „mantra”: arra ösztönzi a fejlesztőket, hogy a biztonságot a DevOps (szállítási) folyamat jobb oldaláról (végéről) a bal oldalára (elejére) tolja át.
- Egy DevSecOps-környezetben a biztonság a fejlesztési folyamat szerves része a kezdetektől fogva.
- A DevSecOps esetén a kiberbiztonsági architektek és a biztonsági mérnökök a fejlesztő csapat szerves része. Feladatuk biztosítani, hogy a „stack” minden eleme és konfigurációs eleme biztonságosan konfigurálva és dokumentálva legyen.
- A Shift left lehetővé teszi, hogy a biztonsági kockázatokat, kitettségeket idejekorán felismerjék, és kezeljék. (a lehető legkorábban)

109

110

„Treath Modelling” Fenyegetettség modellezés

- Egy strukturált megközelítés a rendszert fenyegető biztonsági kockázatok azonosítására, értékelésére és mérséklésére.
- Magában foglalja a lehetséges támadások előrejelzését, hatásuk megértését és az ellenük való védekezés stratégiáinak kidolgozását.
- Ez a proaktív megközelítés biztosítja, hogy a sebezhetőségeket már a fejlesztési folyamat korai szakaszában azonosítsák, csökkentve ezzel a rendszer kitettségét.
- Több különböző modell létezik: STRIDE, PASTA, Thread tree, stb.

110

111

A STRIDE módszer

- különféle kategóriákba sorolja a fenyegetéseket annak alapján, hogy mely biztonsági célokat érintik.
- Az egyes fenyegetések kockázatértékelése során a kockázati szintet külön ki lehet számítani az egyes célkitűzésekhez.
- A fenyegetésmodellezés öt fő lépése:
 1. A biztonsági követelmények meghatározása.
 2. Alkalmazási diagram készítése.
 3. A fenyegetések azonosítása.
 4. A fenyegetések mérséklése.
 5. A fenyegetések mérséklésének validálása.

111

STRIDE treath modell

112

	Fenyegetés	Sérült Tulajdonság	Leírás
S	Spoofing	Authentication	Valaminek vagy valakinek kiadja magát, pl. IP spoofing
T	Tampering	Integrity	A lemezen, hálózaton, memóriában vagy máshol lévő infó módosítása
R	Repudiation	Non-Repudiation	Azt állítani, hogy nem tettünk valamit, vagy nem vagyunk felelősek.
I	Information disclosure	Confidentiality	Információ átadása olyan személynek, aki nem jogosult rá
D	Denial of service	Availability	a szolgáltatás nyújtásához szükséges erőforrások kimerítése
E	Elevation of Privileg	Authorization	engedélyezni valakinek, hogy olyasmit tegyen, amire nincs felhatalmazása

112

A DevSecOps stack

113

- A DevSecOps környezetben a "stack" kifejezés azokat az eszközöket és technológiákat jelenti, amelyeket a fejlesztési, biztonsági és üzemeltetési csapatok használnak a szoftverfejlesztési életciklus során.
- Ez a "stack" magában foglalja azokat az eszközöket, amelyek segítik a biztonság beépítését a folyamatba, az automatizált tesztelést, a biztonsági ellenőrzést és a folyamatos visszajelzést.
- A DevSecOps stack főbb elemei:
 - ❖ Forráskód-kezelés (SCM – Source Code Management)
 - ❖ Folyamatos integráció (CI)
 - ❖ Folyamatos szállítás (CD)

113

➤ A DevSecOps stack főbb elemei (folytatás):

114

❖ Biztonsági tesztelés:

- ❑ SAST (Static Application Security Testing): Statikus kódanalízis, amely a kód biztonsági réseit keresi anélkül, hogy a kódot futtatná.
- ❑ DAST (Dynamic Application Security Testing): Dinamikus alkalmazásbiztonsági tesztelés, amely a futó alkalmazás biztonsági réseit keresi.
- ❑ IAST (Interactive Application Security Testing): Interaktív alkalmazásbiztonsági tesztelés, amely a SAST és a DAST kombinációja.
- ❑ SCA (Software Composition Analysis): nyílt forráskódú komponensek biztonsági réseinek keresése.

114

➤ A DevSecOps stack főbb elemei (folytatás):

115

❖ Konténerbiztonság:

- ❑ Docker Security Scanning, Aqua Security, Twistlock: a konténerek biztonsági réseinek felderítését és javítását segítik.

❖ Felhőbiztonság:

- ❑ AWS Security Hub, Azure Security Center, Google Cloud Security Command Center: a felhőalapú környezetek biztonsági réseinek felderítését és javítását szolgálják.

❖ Infrastruktúra mint kód (IaC) biztonság:

- ❑ Checkov, Terrascan: az IaC fájlok biztonsági réseit keresik.

115

- A DevSecOps stack főbb elemei (folytatás):
 - ❖ Biztonsági információk és eseménykezelés (SIEM - Security Information and Event Management):
 - ❑ Splunk, ELK Stack, Sumo Logic: a biztonsági események naplózására, elemzésére szolgálnak (naplógyűjtés, esemény-korreláció keresés, riasztások kiadása, jelentések készítése).
 - ❖ Automatizált konfigurációkezelés:
 - ❑ Chef, Puppet. segítségükkel automatizálhatjuk a szerverek és alkalmazások konfigurációját.